



|                        |                                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------------|
| Nom i Cognoms:         | Rubén Bellido Navarro                                                                           |
| URL Repositori Github: | <a href="https://github.com/ruben-bena/mp06-pr4.2">https://github.com/ruben-bena/mp06-pr4.2</a> |

## ACTIVITAT

### Objectius:

- Familiaritzar-se amb el desenvolupament d'APIs REST utilitzant Express.js
- Aprendre a integrar serveis de processament de llenguatge natural i visió artificial
- Practicar la implementació de patrons d'accés a dades i gestió de bases de dades
- Desenvolupar habilitats en documentació d'APIs i logging
- Treballar amb formats JSON i processament de dades estructurades

### Criteris d'avaluació:

- Cada pregunta indica la puntuació corresponent

### Entrega:

- Repositori git que contingui el codi que resol els exercicis i, en el directori "doc", aquesta memòria resposta amb nom "memoria.pdf"

### Punt de partida

<https://github.com/jpala4-ieti/DAM-M0486-Tema3-RA6-PR4.2-Punt-Partida-25-26>



## Preparació de l'activitat

- Clonar el repositori de punt de partida
- Llegir els fitxers README\*.md que trobaràs en els diferents directoris
- Assegurar-te de tenir una instància de MySQL/MariaDB funcionant
- Tenir accés a una instància d'Ollama funcionant (al centre te'n facilitem una)

## Entrega

- URL de repositori



# Exercicis

## Exercici 1 (2.5 punts)

L'objectiu de l'exercici és familiaritzar-te amb **xat-api**. Respon la les preguntes dins el requadre que trobaràs al final de l'exercici.

Configuració i Estructura Bàsica:

1. Per què és important organitzar el codi en una estructura de directoris com controllers/, routes/, models/, etc.? Quins avantatges ofereix aquesta organització?
2. Analitzant el fitxer server.js, quina és la seqüència correcta per inicialitzar una aplicació Express? Per què és important l'ordre dels middlewares?
3. Com gestiona el projecte les variables d'entorn? Quins avantatges ofereix usar dotenv respecte a hardcodejar els valors?

1. Es importante organizar el código para categorizar las responsabilidades del software en diferentes lugares, lo cuál es útil cuando hay que modificar o extender esas responsabilidades. Lo contrario dificultará añadir una nueva implementación, modificar el código existente para arreglar un bug, etc.
2. La secuencia correcta es:
  - a. Crear la instancia de Express
  - b. Definir los middlewares.  
El orden es importante porque es el orden en el que Express los va a ejecutar. Si por ejemplo hiciésemos:  

```
app.use('/api/chat', chatRoutes);  
app.use(express.json());
```

Entonces las rutas se ejecutarán antes de que el body sea parseado.
  - c. Definir rutas
  - d. Definir manejo de errores
  - e. Arrancar server
3. El proyecto gestiona las variables de entorno leyéndolas de un archivo .env, que se encarga de registrar en el código.  
Las ventajas de trabajar con dotenv en lugar de valores hardcodeados son:
  - a. Separar código de configuración
  - b. Seguridad a la hora de guardar credenciales (el código se sube al repositorio online, el .env se pone en el .gitignore)
  - c. Plataformas como docker o kubernetes trabajan con archivos .env, no con valores hardcodeados
  - d. Mejora la mantenibilidad del código



### API REST i Express:

1. Observant `chatRoutes.js`, com s'implementa el routing en Express? Quina és la diferència entre els mètodes HTTP GET i POST i quan s'hauria d'usar cadascun?
2. En el fitxer `chatController.js`, per què és important separar la lògica del controlador de les rutes? Quins principis de disseny s'apliquen?
3. Com gestiona el projecte els errors HTTP? Analitza el middleware `errorHandler.js` i explica com centralitza la gestió d'errors.

1. El routing en Express se implementa definiendo a qué lógica tiene que llamar el software cuando se hagan ciertas llamadas a la API.  
Por ejemplo, cuando alguien hace una llamada a `/api/chat/prompt`, se ejecuta lo siguiente:  

```
router.post('/prompt', registerPrompt);
```

  
Es decir, se llama a la función `registerPrompt`.
2. Es importante hacer esta separación para respetar el principio SOLID de responsabilidad única, que implica separar en archivos diferentes cosas que hagan cosas diferentes: `chatRoutes.js` define las rutas y a qué lógica se llama, y `chatController.js` define la lógica en sí misma.
3. Cuando ocurre un error durante la ejecución de lógica en `chatController.js`, todas las funciones hacen catch del error y llaman a `next(error)`; con lo que se pasa al middleware de `errorHandler.js`  
Dependiendo del tipo de error recibido en el código dentro de `errorHandler.js`, se hace una cosa u otra. Hay errores que están controlados como `SequelizeValidationError` y `SequelizeUniqueConstraintError`, y el resto de errores se tratan de forma genérica todos ellos.



Documentació amb Swagger:

1. Observant la configuració de Swagger a `swagger.js` i els comentaris a `chatRoutes.js`, com s'integra la documentació amb el codi? Quins beneficis aporta aquesta aproximació?
2. Com es documenten els diferents endpoints amb els decoradors de Swagger? Per què és important documentar els paràmetres d'entrada i sortida?
3. Com podem provar els endpoints directament des de la interfície de Swagger? Quins avantatges ofereix això durant el desenvolupament?

1. En [swagger.js](#) se define la especificación de Swagger, y en [chatRoutes.js](#) se encuentran los comentarios con `@Swagger` que permiten generar la documentación de forma automática a partir del código. Hacer una documentación de esta manera hace que la documentación esté junto al código, que cualquiera pueda entender y probar la API, y estamos utilizando un estándar profesional (OpenAPI).
2. Se documentan siguiendo el estándar de Swagger, y es importante documentar los parámetros de entrada y de salida porque son el contrato de la API.
3. Arrancando el servidor, tendremos acceso a la interfaz de Swagger desde el navegador, conectándonos a <http://localhost:3000/api-docs>, y podremos probar directamente las diferentes llamadas de la API.  
Las ventajas que tiene usar Swagger durante el desarrollo son:
  - a. Podemos probar los cambios en el código de forma rápida, recargando Swagger tras reiniciar el server con los cambios
  - b. Menor dependencia del frontend
  - c. Facilita el trabajo en equipo ya que todos pueden probar la API fácilmente



### Base de Dades i Models:

1. Analitzant els models Conversation.js i Prompt.js, com s'implementen les relacions entre models utilitzant Sequelize? Per què s'utilitza UUID com a clau primària?
2. Com gestiona el projecte les migracions i sincronització de la base de dades? Quins riscos té usar `sync()` en producció?
3. Quins avantatges ofereix usar un ORM com Sequelize respecte a fer consultes SQL directes?

1. Si miramos [index.js](#), podemos ver que la relación entre Conversation y Prompt es de 1:N (una conversación tiene muchos prompts).  
Se utiliza UUID como tipo de dato en la clave primaria porque:
  - a. Tiene unicidad global
  - b. Diferentes servicios pueden generar estos ID sin conflicto
  - c. Se generan fuera de la BBDDEstas cualidades hacen que sea más deseable que el típico valor auto-incremental para identificar tablas en una BBDD.
2. En este proyecto no se gestionan las migraciones, y la sincronización se gestiona de forma automática con Sequelize.  
Los riesgos de utilizar `sync()` en producción son:
  - a. Perder datos (con `force: true`)
  - b. Inconsistencias de esquema (con `alter: true`)
  - c. Si hay multiples instancias de BBDD, puede haber conflictos usando `sync()`
3. Las principales ventajas de utilizar un ORM son:
  - a. Abstracción del SQL y trabajar con objetos y métodos de JS
  - b. Gestión de modelos y relaciones sin hacer JOIN manualmente
  - c. Compatibilidad con diferentes BBDD
  - d. Se pueden definir validaciones y restricciones a los modelos
  - e. Facilita migraciones y sincronización



### Logging i Monitorització:

1. Observant `logger.js`, com s'implementa el logging estructurat? Quins nivells de logging existeixen i quan s'hauria d'usar cadascun?
2. Per què és important tenir diferents transports de logging (consola, fitxer)? Com es configuren en el projecte?
3. Com ajuda el logging a debugar problemes en producció? Quina informació crítica s'hauria de loguejar?

1. El logging estructurado se implementa con Winston, y cada registro contiene:

- a. timestamp
- b. level (info, debug, error)
- c. message
- d. stack
- e. metadata

Los niveles de logging que existen son:

- f. Info (información de la ejecución del software que siempre quieres registrar)
- g. Debug (información pensada para que la vean los desarrolladores y que les ayude a visualizar el comportamiento del software)
- h. Error (información crítica relacionada con los fallos que ocurren en el software, ya sean intencionales como una llamada mal hecha, o inesperados)

2. Es importante tener diferentes transportes de logging porque cada uno cumple una función importante:

- a. Consola → Feedback inmediato, útil durante el desarrollo
- b. Fichero → Historial persistente, útil para producción y auditorías

En el proyecto está configurado dentro de `logger.js`, donde hay una configuración específica definida en caso de que exista un path para logs en el `dotenv`.

3. El logging ayuda a debugar problemas en producción siguiendo el flujo del programa e imprimiendo información relevante, ya sea del correcto comportamiento de la lógica de negocio o dando información de los errores que se producen utilizando un stacktrace sobre los diferentes errores que tenemos contemplados en el código.



## Exercici 2 (2.5 punts)

Dins de **practica-codi** trobaràs **src/exercici2.js**

Modifica el codi per tal que, pels dos primers jocs i les 2 primeres reviews de cada joc, crei una estadística que indiqui el nombre de reviews positives, negatives o neutres.

Modifica el prompt si cal.

Guarda la sortida en el directori data amb el nom **exercici2\_resposta.json**

Exemple de sortida

```
{
  "timestamp": "2025-01-09T12:30:45.678Z",
  "games": [
    {
      "appid": "730",
      "name": "Counter-Strike 2",
      "statistics": {
        "positive": 1,
        "negative": 0,
        "neutral": 1,
        "error": 0
      }
    },
    {
      "appid": "570",
      "name": "Dota 2",
      "statistics": {
        "positive": 1,
        "negative": 1,
        "neutral": 0,
        "error": 0
      }
    }
  ]
}
```





## Exercici 3 (2.5 punts)

Dins de **practica-codi** trobaràs **src/exercici3.js**

Modifica el codi per tal que retorni un anàlisi detallat sobre l'animal.

Modifica el prompt si cal.

La informació que volem obtenir és:

- Nom de l'animal.
- Classificació taxonòmica (mamífer, au, rèptil, etc.)
- Hàbitat natural
- Dieta
- Característiques físiques (mida, color, trets distintius)
- Estat de conservació

Guarda la sortida en el directori **data** amb el nom **exercici3\_resposta.json**

```
{
  "analisi": [
    {
      "imatge": {
        "nom_fitxer": "nom_del_fitxer.jpg",
      },
      "analisi": {
        "nom_comu": "nom comú de l'animal",
        "nom_cientific": "nom científic si és conegut",
        "taxonomia": {
          "classe": "mamífer/au/rèptil/amfibi/peix",
          "ordre": "ordre taxonòmic",
          "familia": "família taxonòmica"
        },
        "habitat": {
          "tipus": ["tipus d'hàbitats"],
          "regioGeografica": ["regions on viu"],
          "clima": ["tipus de climes"]
        },
        "dieta": {
          "tipus": "carnívor/herbívor/omnívor",
          "aliments_principals": ["llista d'aliments"]
        },
        "caracteristiques_fisiques": {
          "mida": {
            "altura_mitjana_cm": "altura mitjana",
            "pes_mitja_kg": "pes mitjà"
          },
          "colors_predominants": ["colors"],
          "trets_distintius": ["característiques"]
        },
        "estat_conservacio": {
          "classificacio_IUCN": "estat",
          "amenaces_principals": ["amenaces"]
        }
      }
    }
  ]
}
```



## Exercici 4 (2.5 punts)

Implementa un nou endpoint a xat-api per realitzar anàlisi de sentiment

Haurà de complir els següents requisits

- Estar disponible a l'endpoint POST /api/chat/sentiment-analysis
- Disposar de documentació swagger
- Emmagatzemar informació a la base de dades
- Usar el logger a fitxer

Abans d'implementar la tasca, explica en el requadre com la plantejaràs i fes una proposta de json d'entrada, de sortida i de com emmagatzemaràs la informació a la base de dades.